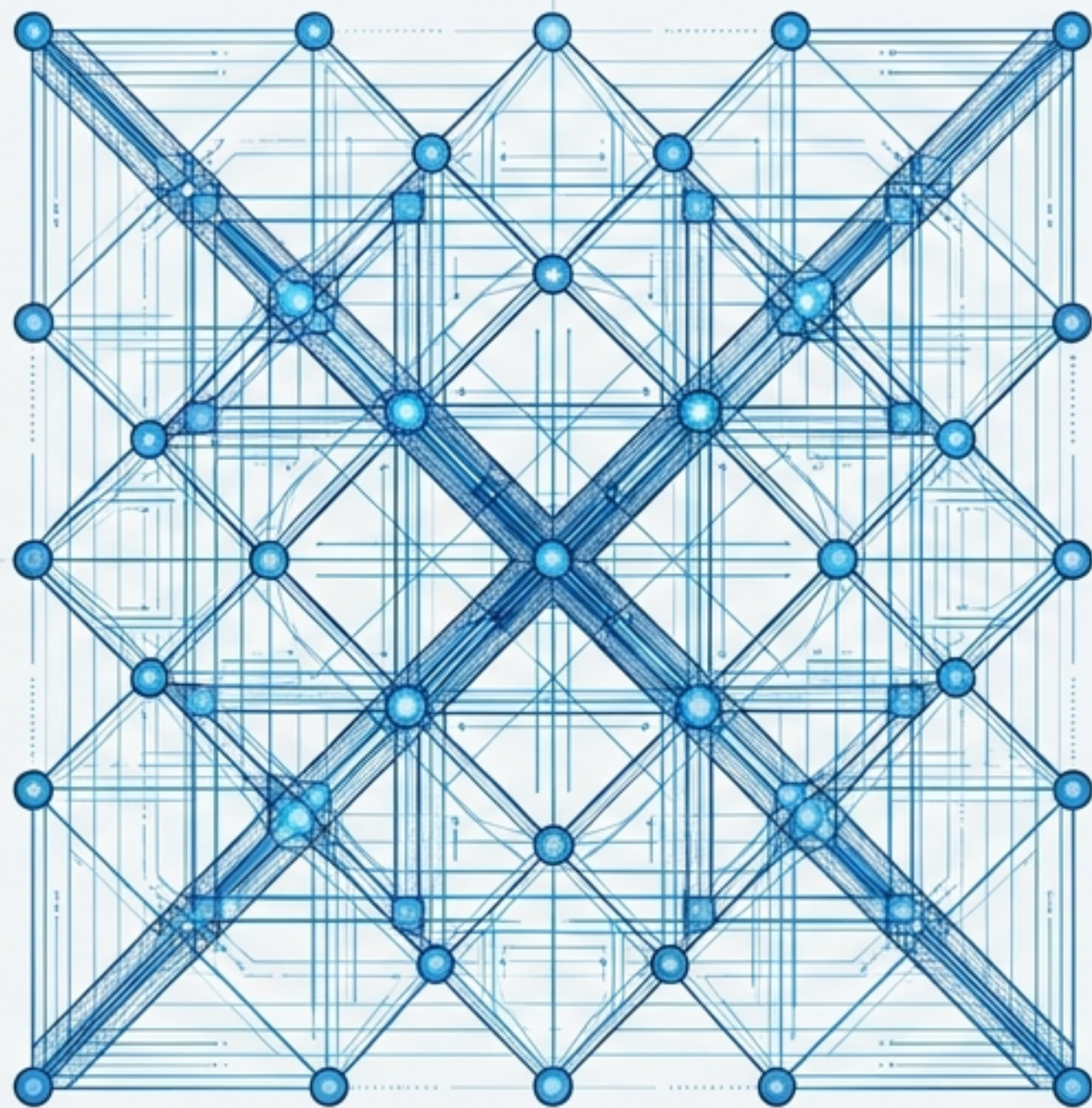




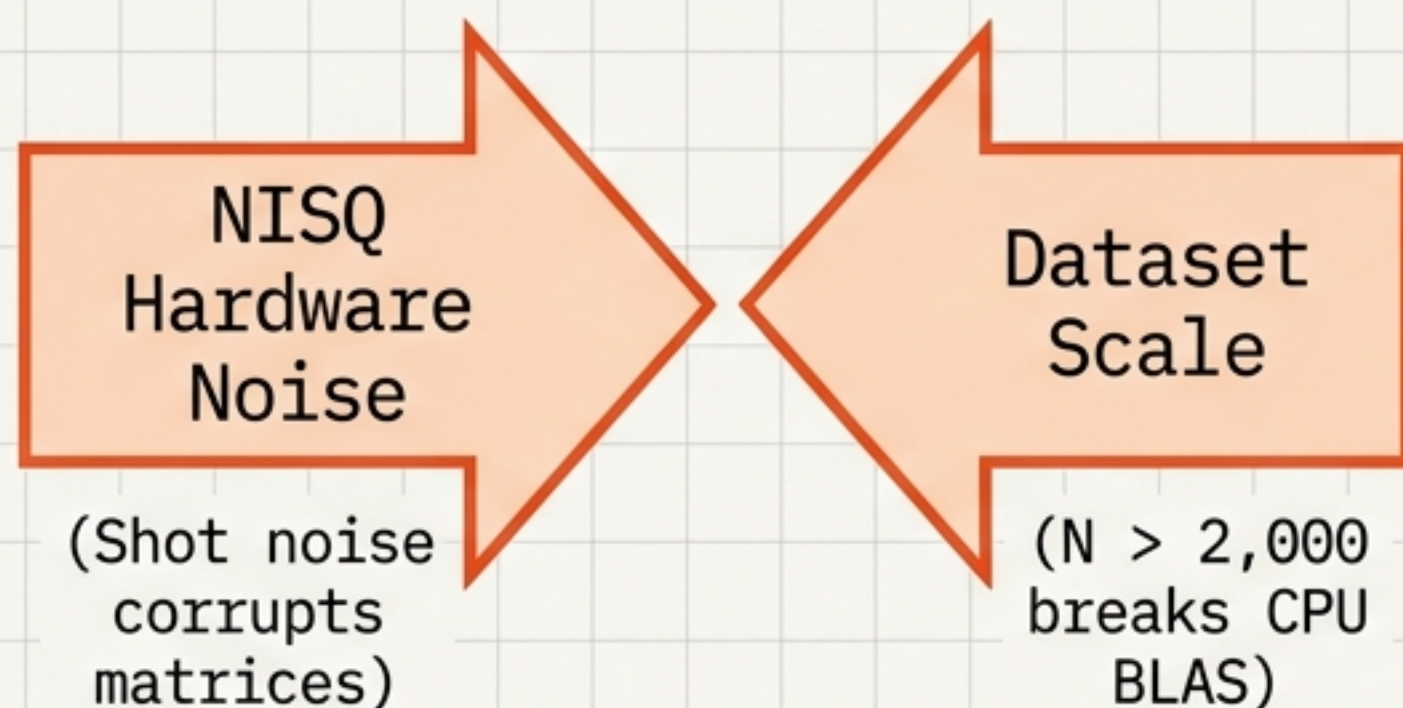
The Hidden Bottleneck in Quantum Machine Learning

How MPDOK eliminates the classical $O(N^3)$ **linear solve barrier** for Quantum Support Vector Machines and Kernel Ridge Regression.

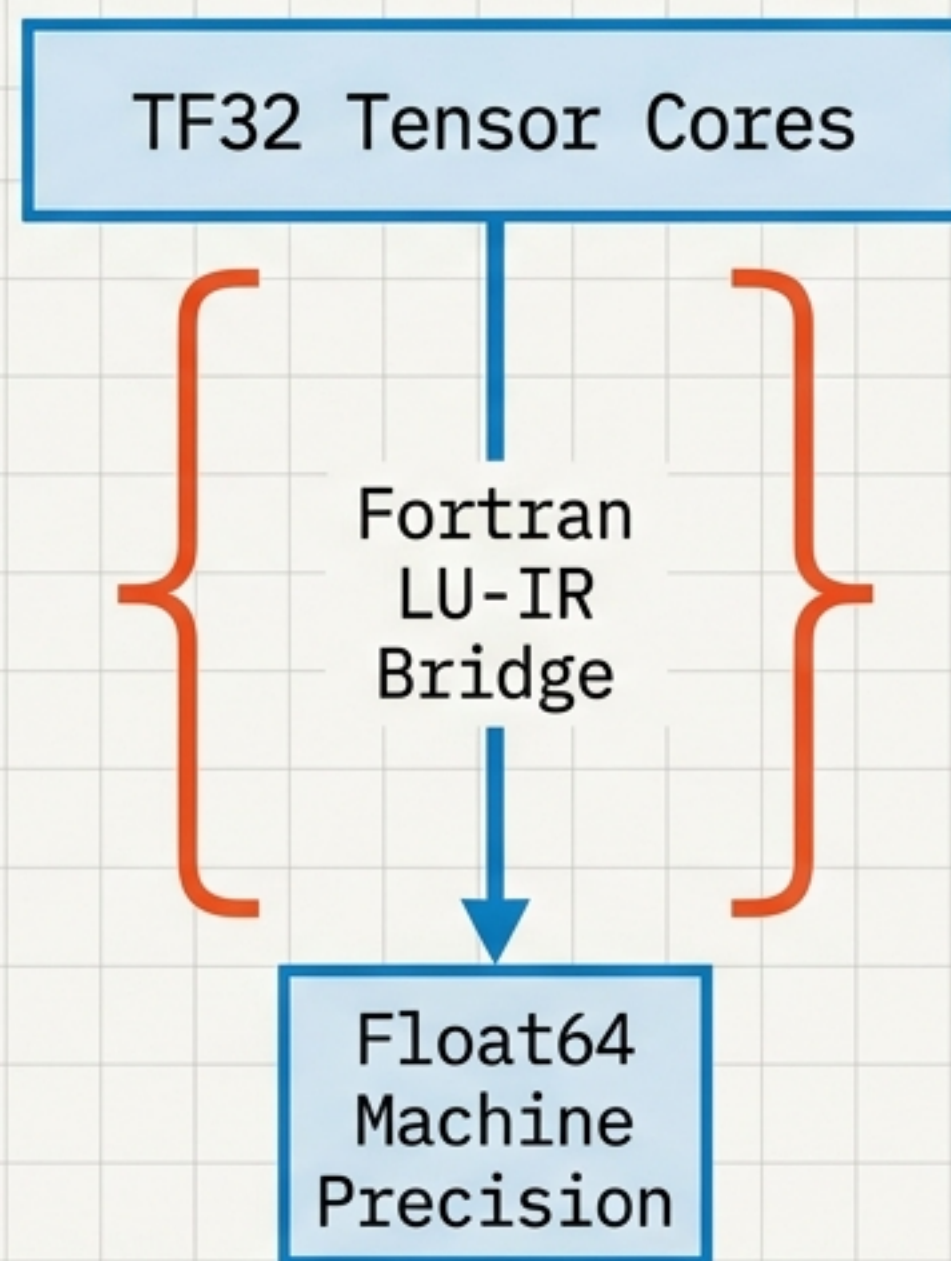
Building a quantum kernel is hard. Solving the **dense linear system** on **classical CPUs** is what causes production systems to choke at scale.

Executive Summary: The Dual Threat in Near-Term QML

The Dual Threat



The Mathematical Blueprint



The Proof

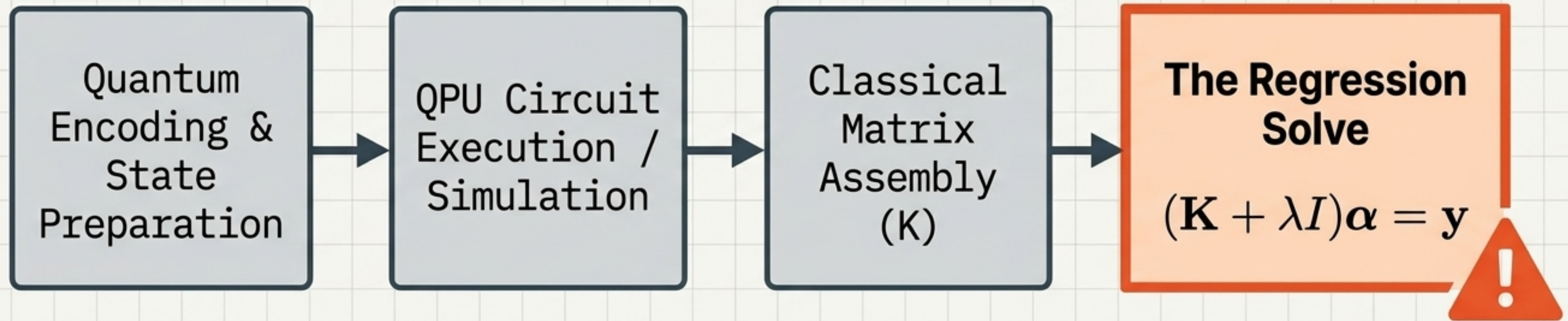
Challenge 1:

Survives **50-shot NISQ noise** at 10^{-14} precision.

Challenge 2:

9.4x faster than NumPy at N=8,000.

The Structural Reality of QML Pipelines



QSVM and QKRR always reduce to this final classical dense linear solve.

Standard backends (sklearn, numpy, scipy) rely on pure float64 CPU BLAS.

CPU solvers scale at $O(N^3)$.

A system that takes milliseconds at $N=500$ balloons to system-crashing hours at production scales.

The MPDOK Solution: Precision meets Tensor Speed

TF32 Tensor Cores

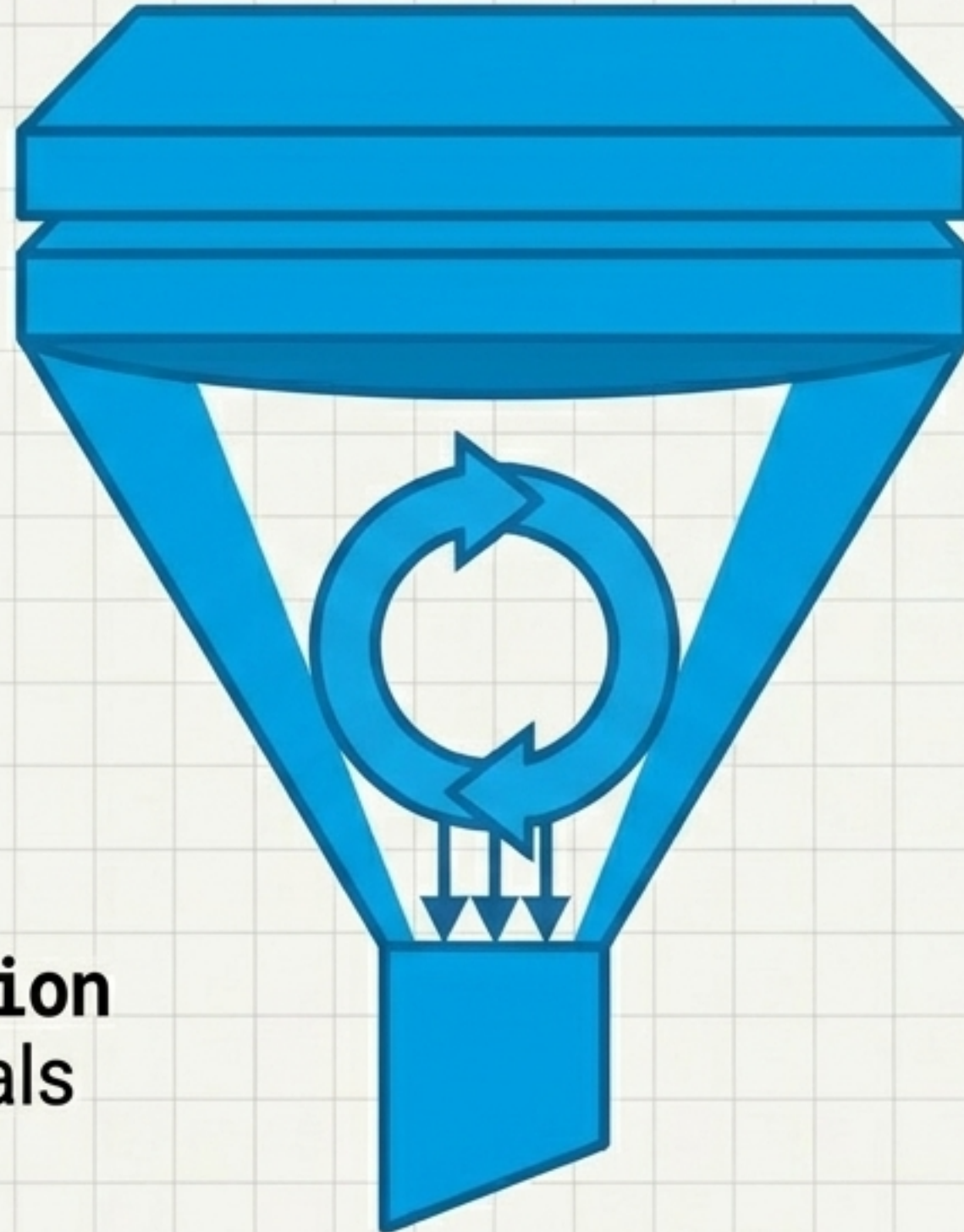
Matrix factorization at 10-bit mantissa precision. $\approx 10^3\times$ faster throughput on RTX architectures.

LU Iterative Refinement (LU-IR)

Recursive loop refining the residual error.

Float64 Machine Precision

Consistent drop-out residuals at 10^{-12} to 10^{-14} .

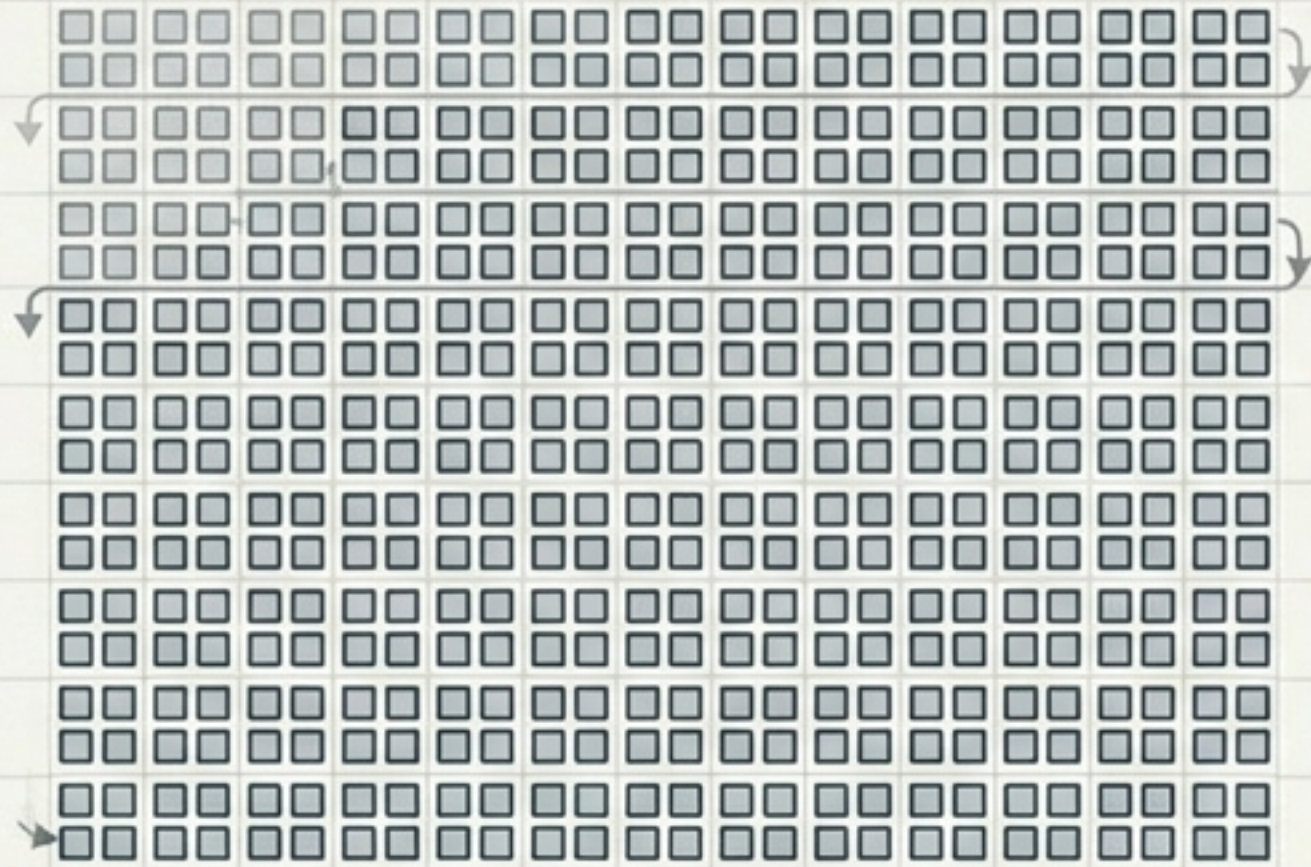


Speed without compromise.

By factorizing in TF32 and refining to float64, MPDOK delivers tensor-core throughput with strict LAPACK-class accuracy.

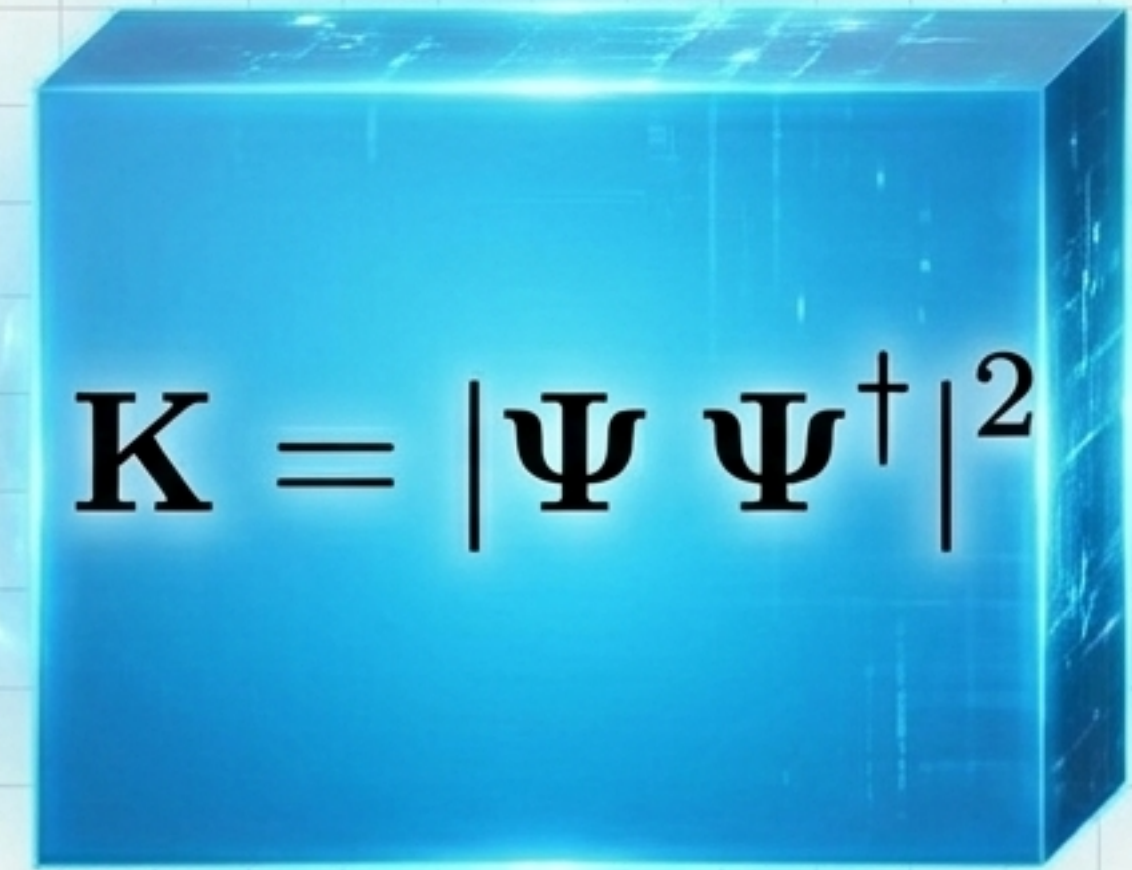
Pre-Solve Math: Eliminating the $O(N^2)$ Build Loop

Naive Approach



Naive Approach: 50 Million Python Calls = Hours of Wall Time

MPDOK Approach



Fidelity Kernel Insight: 1 Matrix Multiplication = 1.4s GPU Pulse

Because the IQP kernel is a fidelity kernel, the full $N \times N$ Gram matrix collapses into a single batched **cuBLAS ZGEMM** inner product. An **800 MB float64** matrix is generated in just **1.4 seconds**.

Diagnostic Matrix: CPU BLAS vs. MPDOK GPU

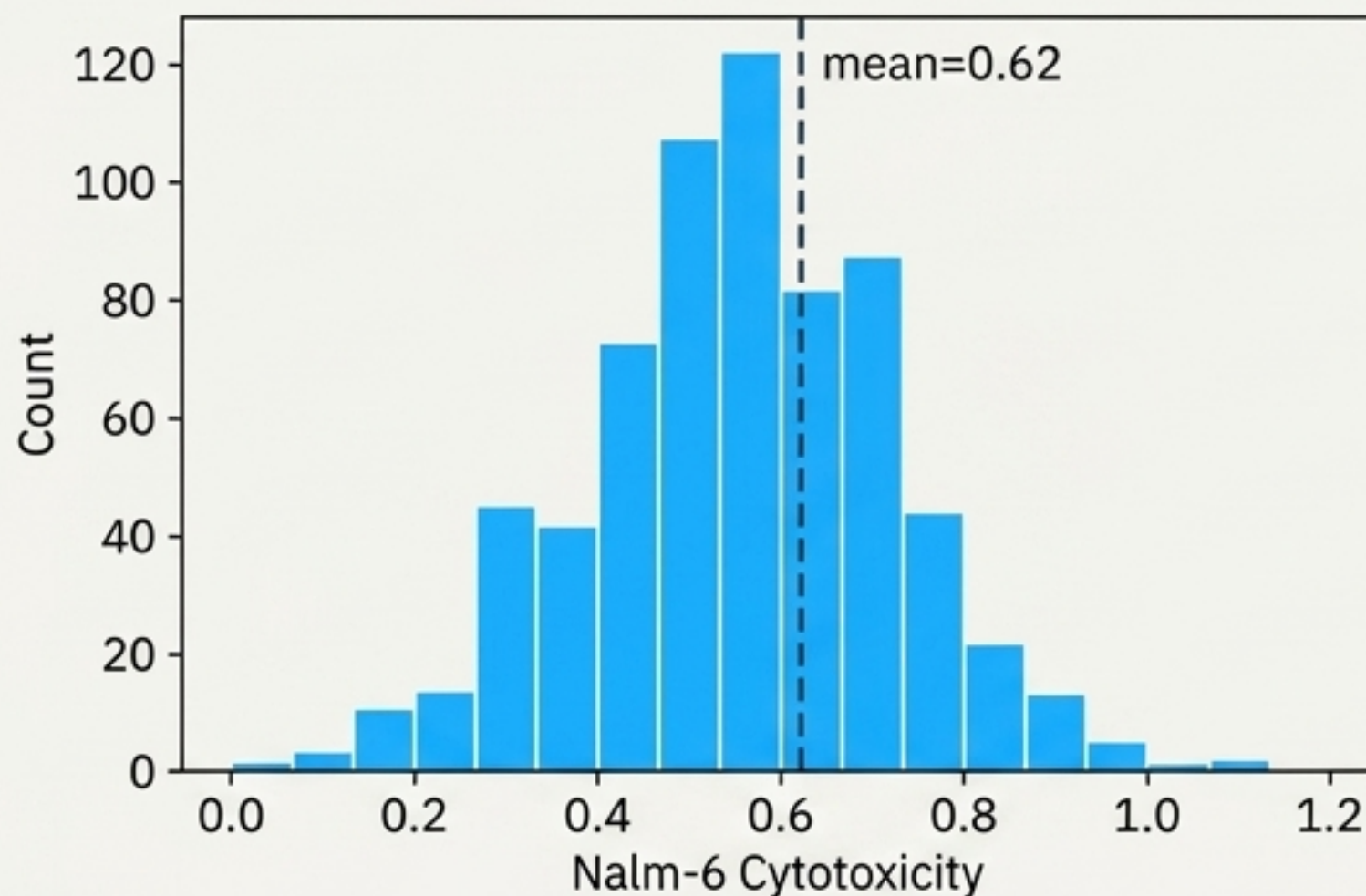
Dataset Scale	CPU BLAS (numpy/scipy)	MPDOK (GPU)	Verdict
Small ($N < 2,000$)	Sub-130ms execution	Burdened by 10-25ms GPU launch cost	CPU Wins
Medium ($N = 2k-10k$)	Cache hierarchy limits throughput	Up to 9.4× faster via Tensor Cores	MPDOK Breakeven Point
Large ($N = 10k-50k+$)	Triggers Out-Of-Memory (OOM)	<code>alloc_managed()</code> utilizes system RAM	MPDOK Only Path

MPDOK is engineered specifically for the medium-to-large scale where production-grade quantum ML experiments operate.

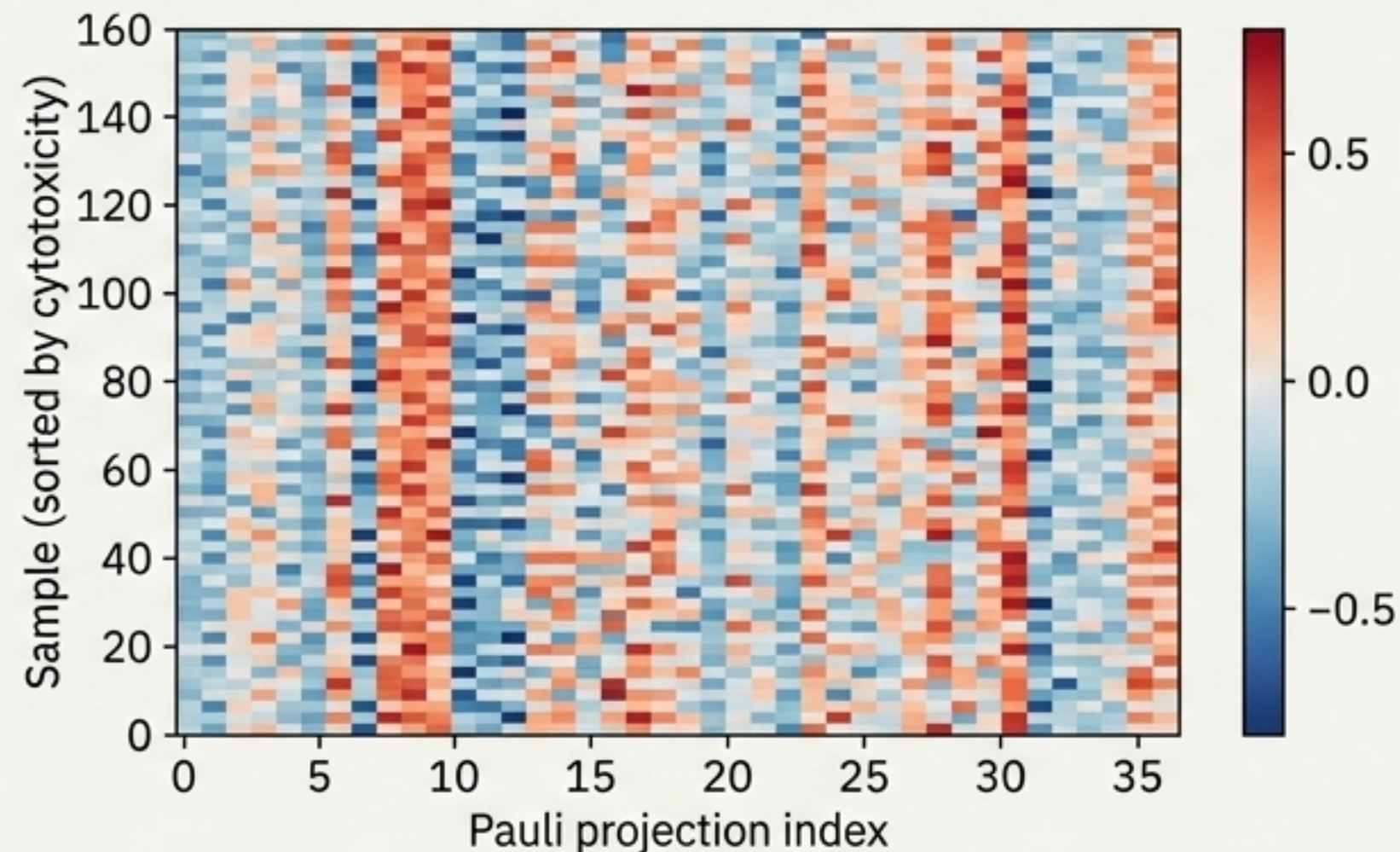
Challenge 1: NISQ Noise Reality

CAR T-cell cytotoxicity experiment using real IBM quantum hardware (N=172)

A • Target Distribution



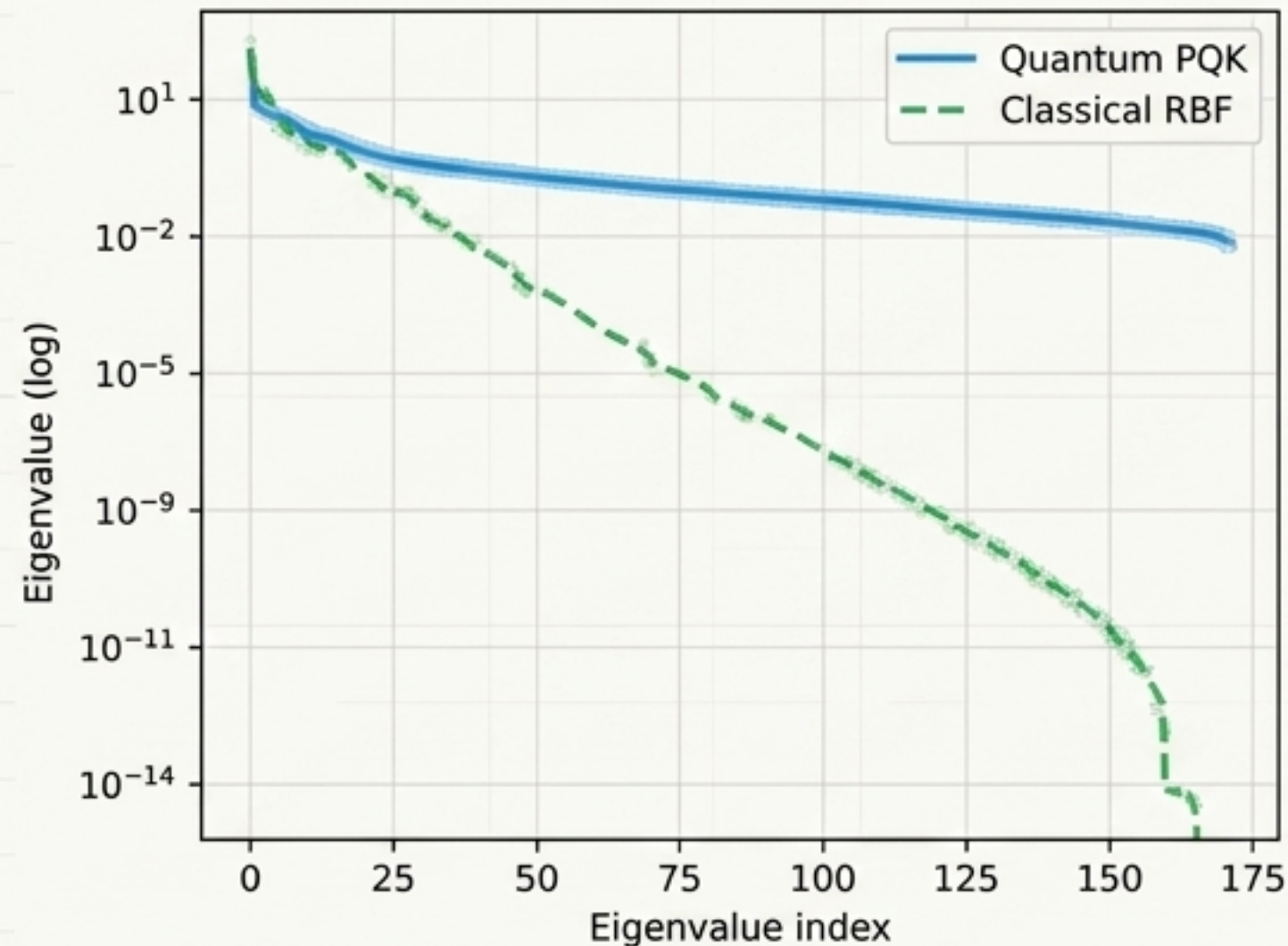
B • IBM Quantum Projections $\langle \sigma_k \rangle$ (real hardware, 180-dim)



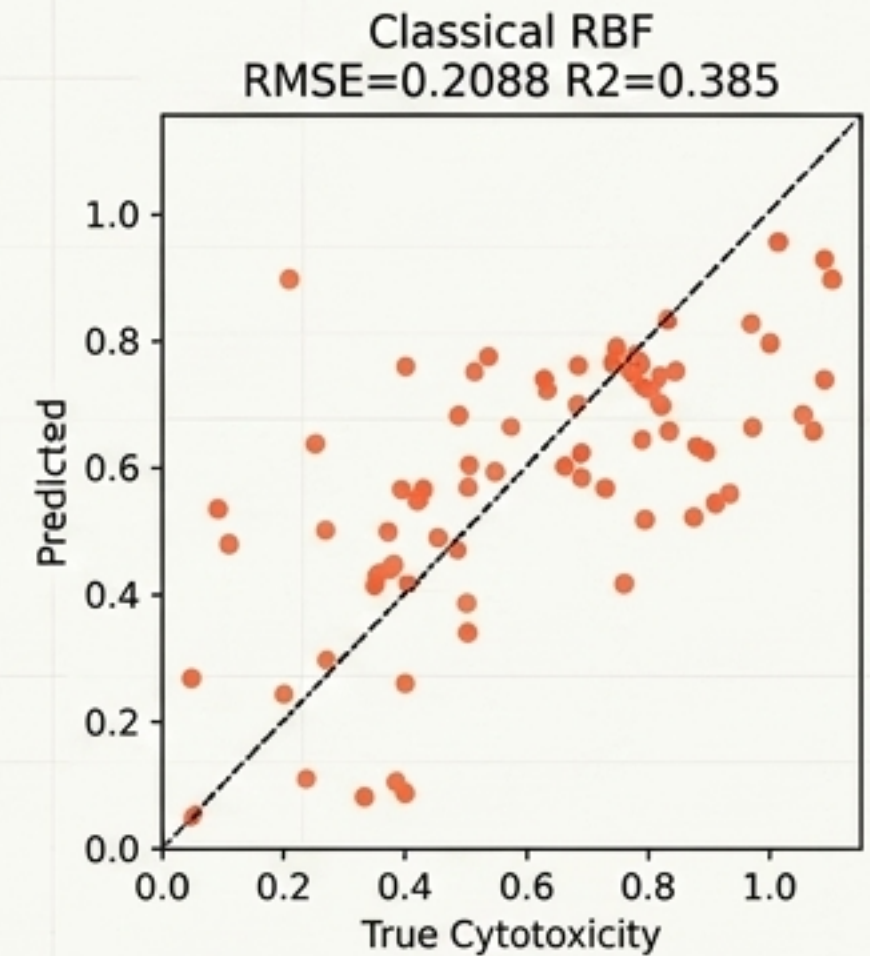
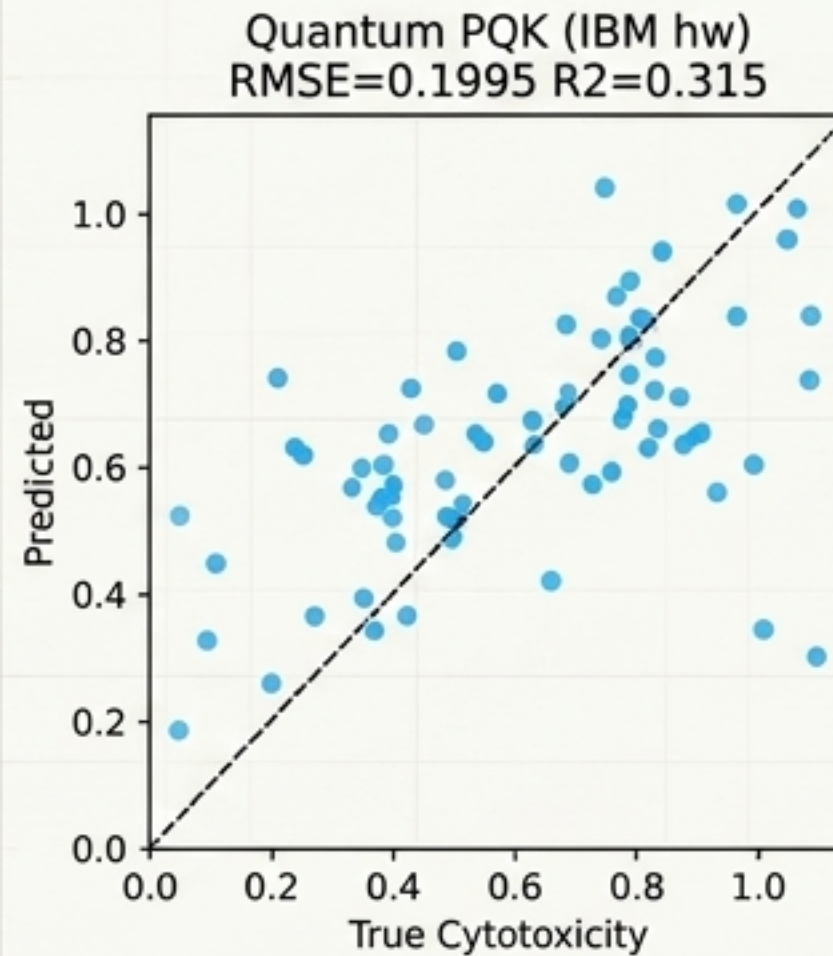
The Hardware Reality: IBM hardware does not output a ready-made kernel matrix. It outputs raw, noisy projection vectors (ϕ) which must be classically assembled into a **Projected Quantum Kernel** (PQK).

Intrinsic Value: Quantum Kernel Superiority

Superior Conditioning: Quantum PQK (1.5×10^4)
vs Classical RBF (1.05×10^{17})



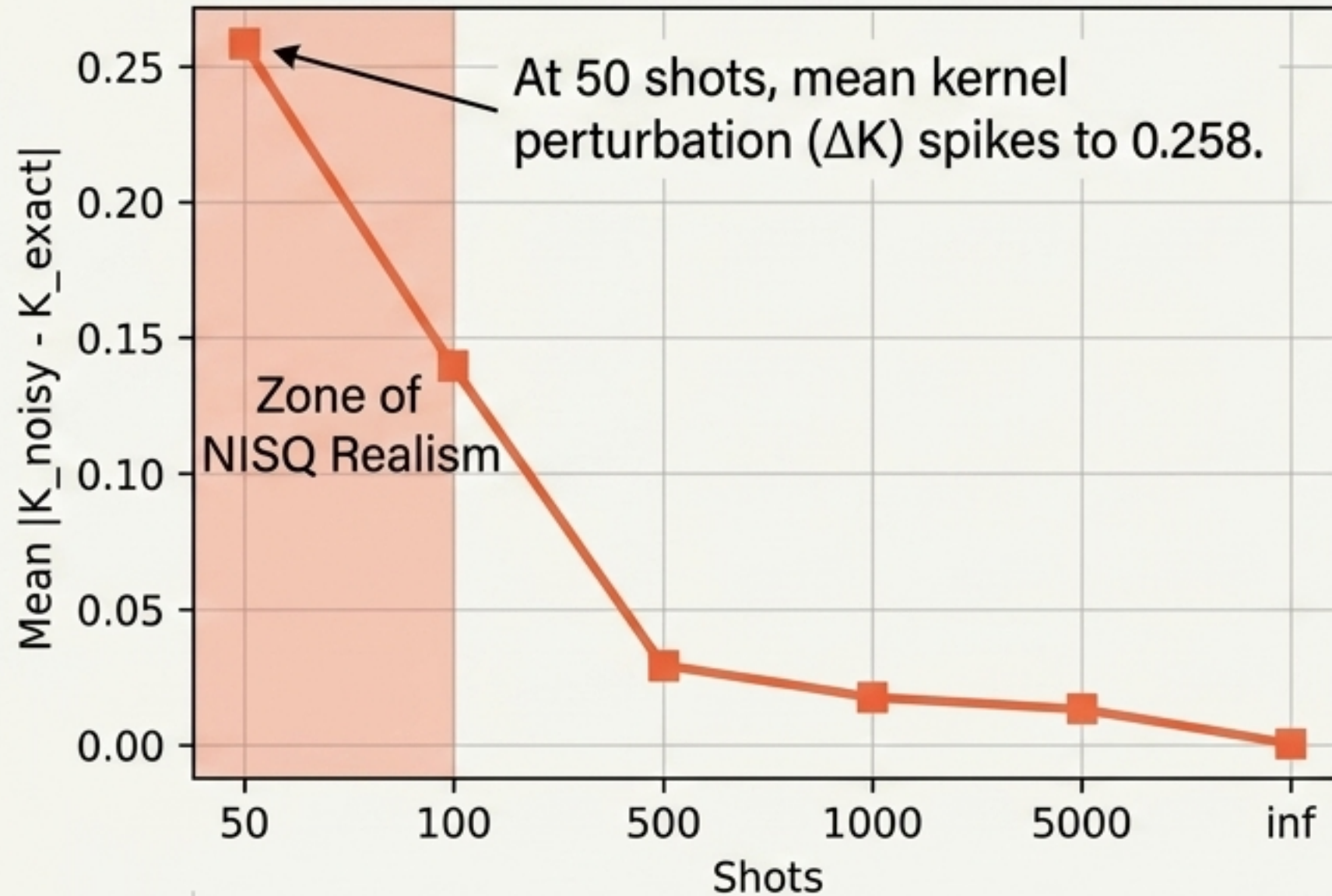
Superior Prediction: Quantum PQK (RMSE 0.1995)
outperforms Classical RBF (RMSE 0.2088)



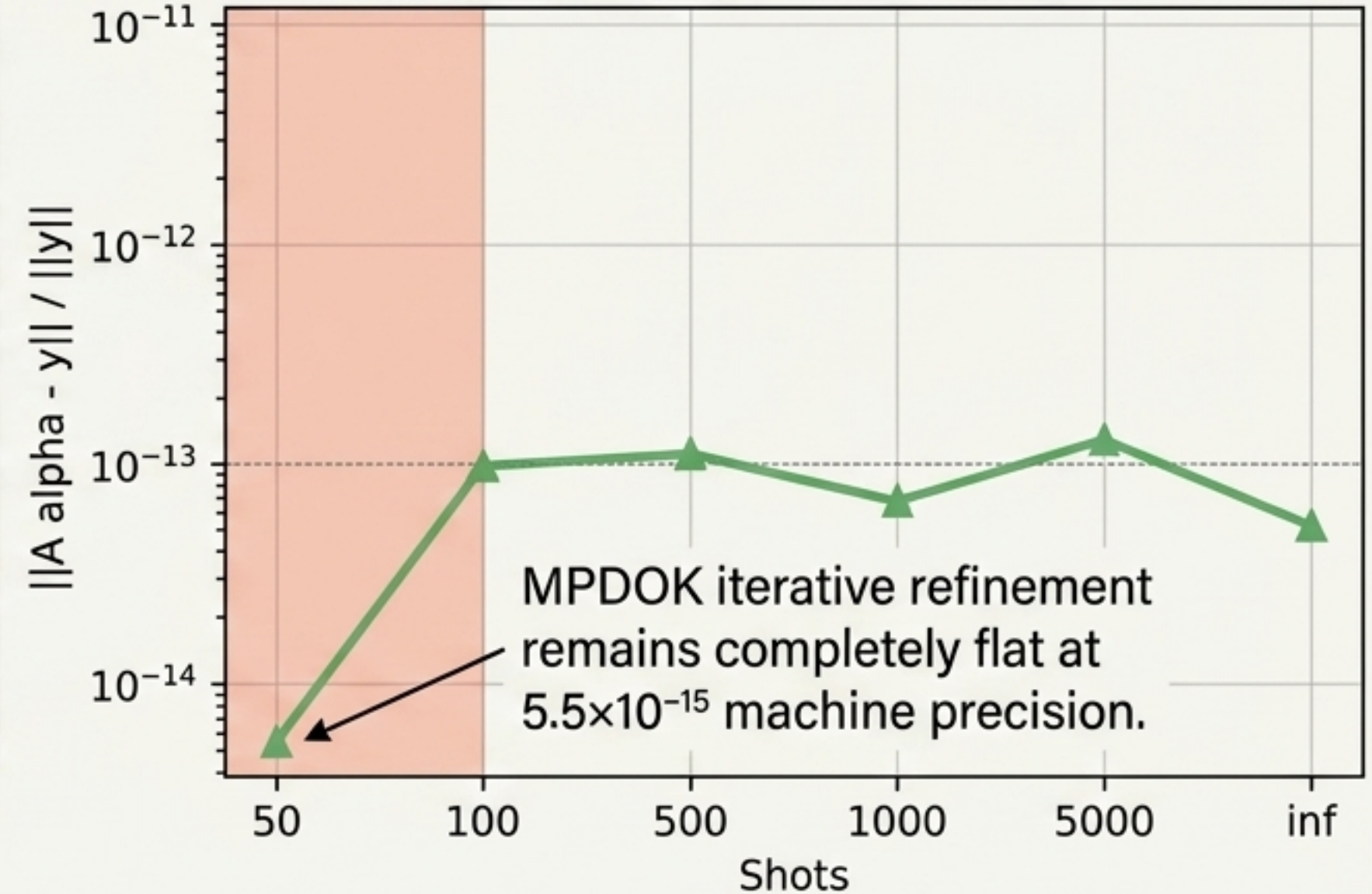
The quantum feature map successfully compresses the discrete signaling domain into a bounded, biologically relevant geometry.

Surviving the Noise: MPDOK's Robustness

Kernel Perturbation vs Shot Noise
(adapted from reference image 0)



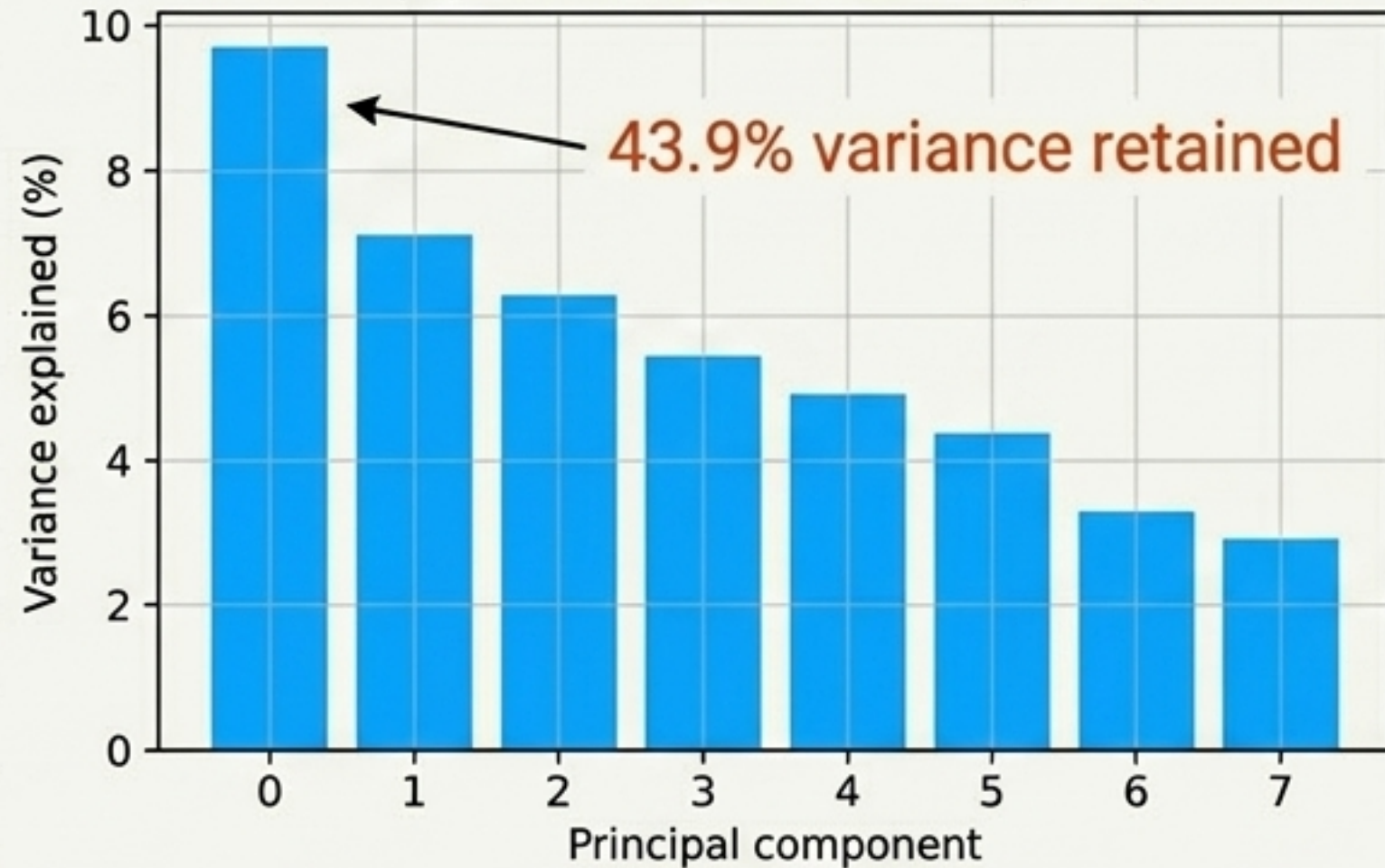
MPDOK Residual
(machine precision despite noisy K)
(adapted from reference image)



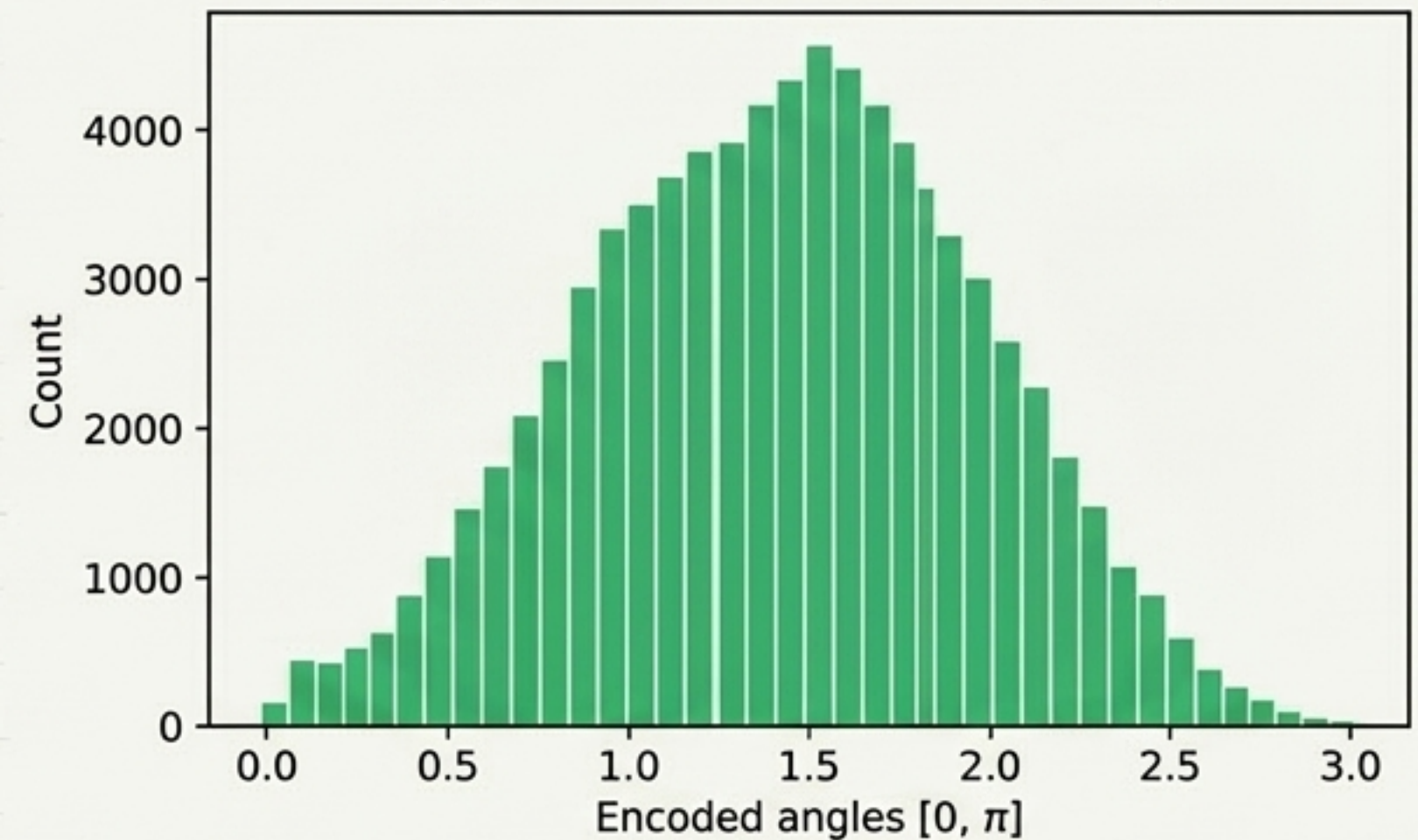
Key Insight: Prediction RMSE degrades gracefully because the bottleneck is quantum measurement noise, **NOT** numerical instability in the classical linear solver.

Challenge 2: Dataset Scale (MNIST Lab)

A · PCA: 784→8 Compression
(reproduced from <IMAGE-2>, plot A)

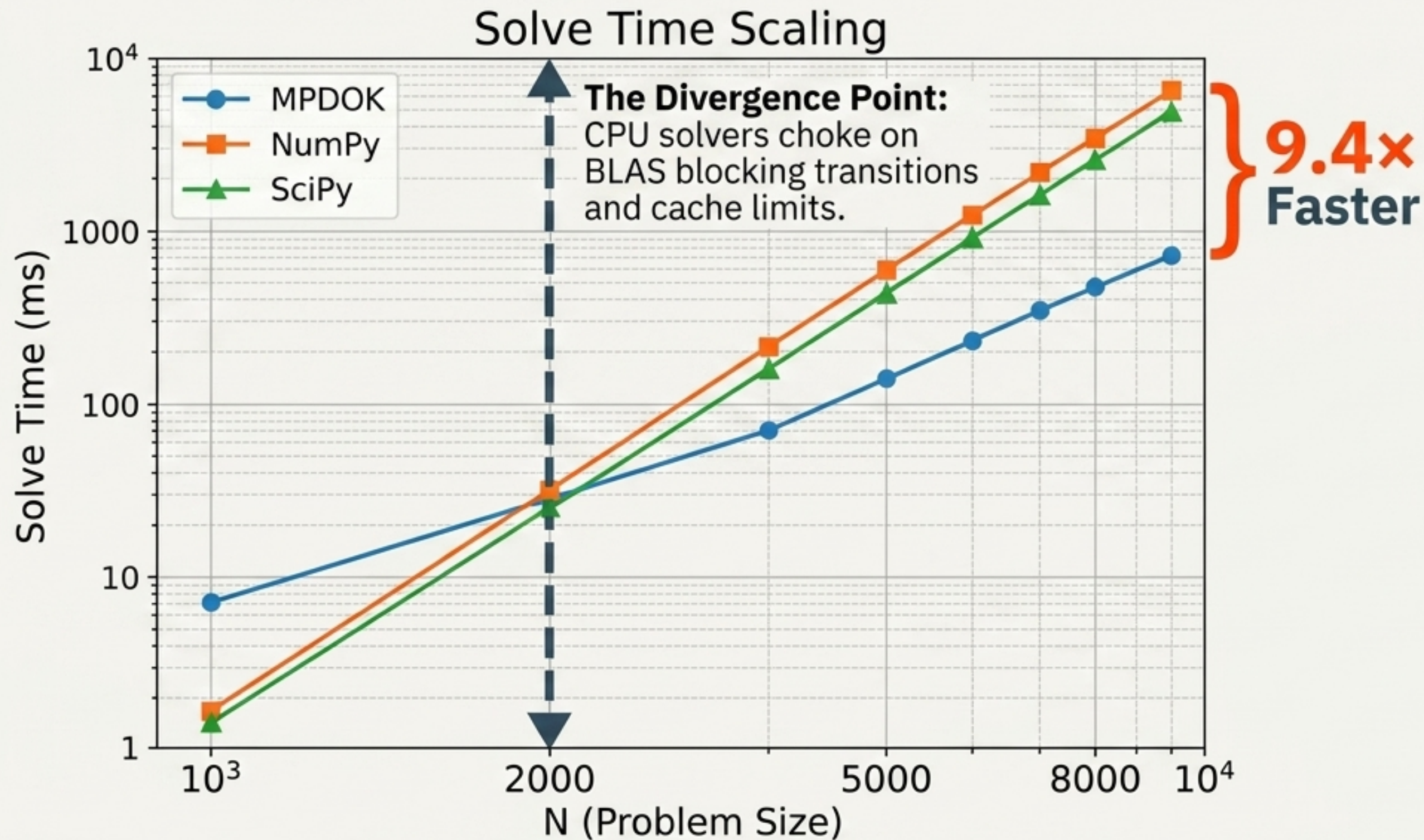


B · Distribution of Encoded Angles
(reproduced from <IMAGE-2>, plot C)



Dataset:	10,000 Training samples, 5,000 Test samples. (Even vs. Odd digits)
Pipeline:	PCA compression → Angle Encoding → 8-qubit IQP Circuit.
System Load:	Statevector size of $2^8 = 256$ complex entries, resulting in an 800 MB Gram matrix held entirely in GPU VRAM.

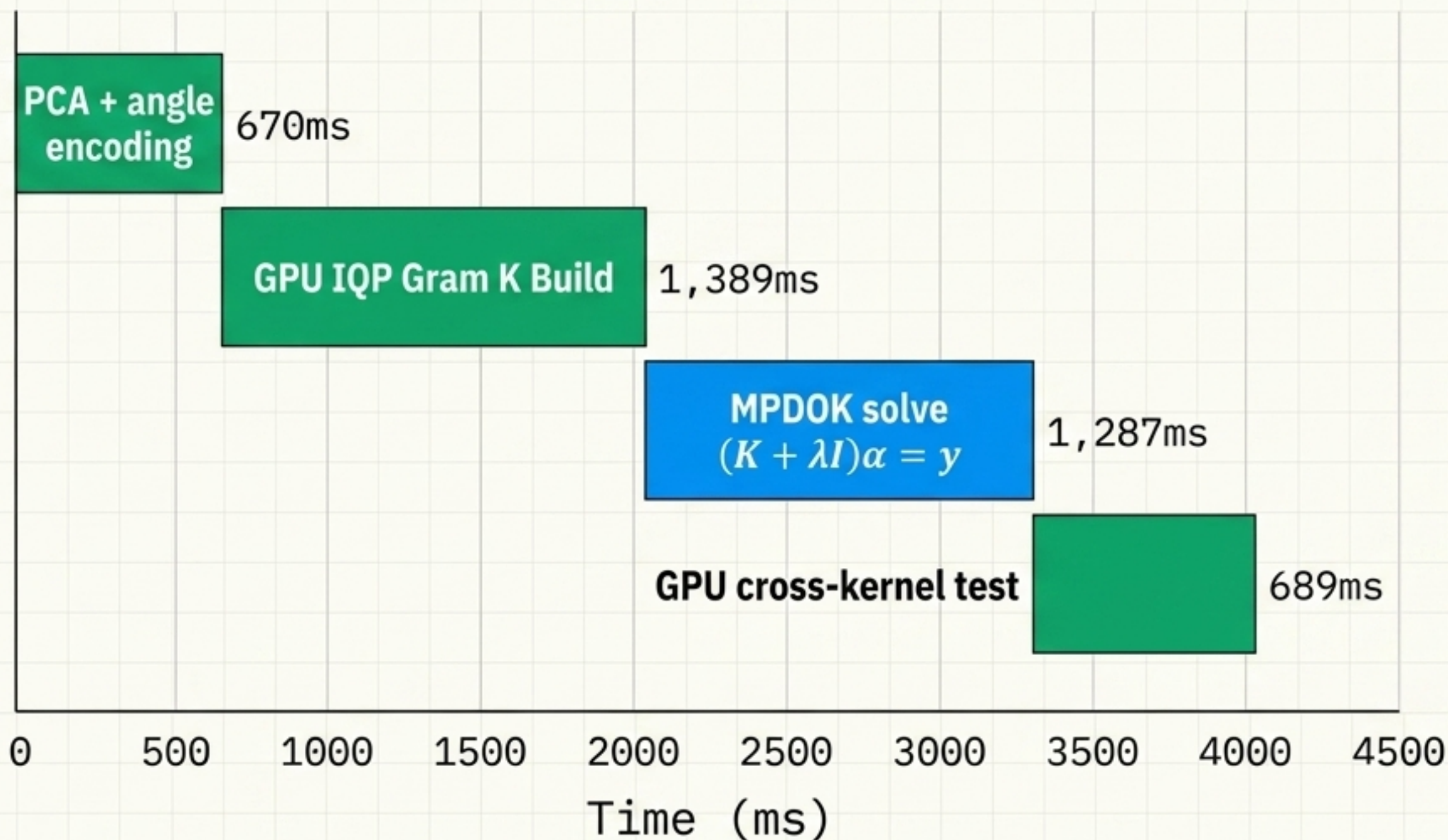
Breaking the Time Barrier at N=10,000



At N=8,000, MPDOK completes the solve in 708ms—crushing NumPy (6,639ms) and SciPy.

Throughout the entire scale curve, residuals remain rigidly locked at 10^{-13} .

End-to-End Pipeline Context



Total execution: ~4.03 seconds.

A complete quantum ML regression pipeline of an 800 MB dense matrix at $N=10,000$ runs in 4 seconds on a consumer GPU.

Classical computation is no longer an excuse.

Technical Assessment: Capabilities & Limits

Small N Overhead

Breakeven is $N \approx 2,000$. For small training sets, the fixed 10-25ms GPU launch cost makes CPU BLAS the correct choice.

Complexity Bound

LU factorization remains fundamentally $O(N^3)$. MPDOK immensely accelerates the constant factor, but does not cheat algorithmic complexity.

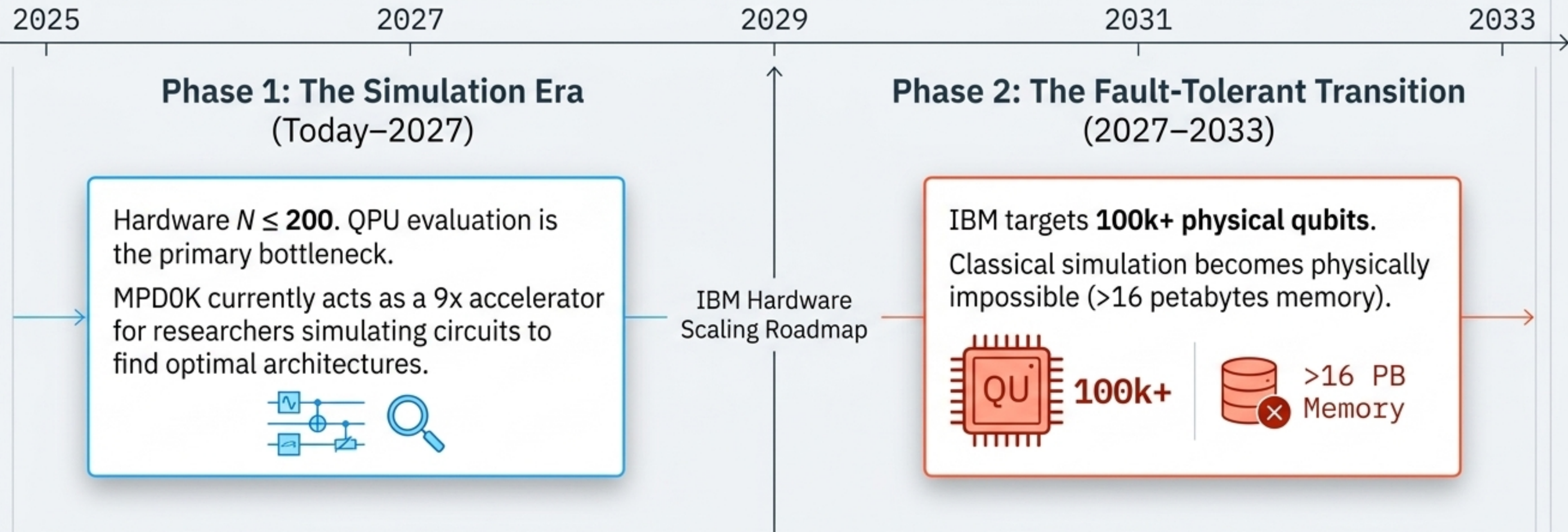
Hardware Ceilings

Single-GPU implementation. Even with managed memory, physical memory ceilings are hit around $N=50,000$. Multi-GPU is the next necessary frontier.

Circuit Physics

The solver cannot fix bad circuit design. The 8-qubit IQP kernel drops 56% of MNIST variance. Better ansätze and higher qubits are fundamentally required.

Implications for the Fault-Tolerant Transition



As hardware speed finally allows datasets to reach $N = 10,000$, ⌚ QPU hours 📊 will become intensely expensive. When K construction takes hours on a physical quantum device, a robust, $O(N^3)$ classical solver ⚙️ guaranteeing machine precision under high shot noise transitions from a **convenience** to a **strict necessity**.

The Essential Plumbing for Quantum Machine Learning.

Before the winning architectures of deep learning were known, the GPU became its unquestioned substrate. MPDOK provides the exact same robust, dense linear infrastructure for the quantum era—ready to solve any matrix, at scale, regardless of the circuit that built it.



Open-Source



TF32 Tensor Core Speed



Ready for $N > 10,000$